K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Batch: B2	**Roll No.: 16010124107**
Experiment No. 06	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Title: Implementation of Single Source shortest path using Dynamic Programming

Objective: To learn Single Source shortest path (Bellman Ford Algorithm) using Dynamic Programming Approach

CO to be achieved:

CO 2 Describe various algorithm design strategies to solve different problems and analyse Complexity.

Books/ Journals/ Websites referred:

1. **Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press**
2. **T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algortihtms",2nd Edition ,MIT press/McGraw Hill,2001**
3. **http://www.lsi.upc.edu/~mjserna/docencia/algofib/P07/dynprog.pdf**
4. **http://www.geeksforgeeks.org/travelling-salesman-problem-set-1/**
5. **http://www.mafy.lut.fi/study/DiscreteOpt/tspdp.pdf**
6. **https://class.coursera.org/algo2-2012-001/lecture/181**
7. **http://www.quora.com/Algorithms/How-do-I-solve-the-travelling-salesman-problem-using-Dynamic-programming**
8. **www.cse.hcmut.edu.vn/~dtanh/download/Appendix_B_2.ppt**
9. **www.ms.unimelb.edu.au/~s620261/powerpoint/chapter9_4.ppt**

Pre Lab/ Prior Concepts:

Data structures, Concepts of algorithm analysis

Historical Profile:

Dynamic Programming (DP) is used heavily in optimization problems (finding the maximum and the minimum of something). Applications range from financial models and operation research to biology and basic algorithm research. So the good news is that

understanding DP is profitable. However, the bad news is that DP is not an algorithm or a data structure that you can memorize. It is a powerful algorithmic design technique.

New Concepts to be learned:

Application of algorithmic design strategy to any problem, dynamic Programming method of problem solving vs other methods of problem solving, optimality of the solution, Single Shortest Path Problems and their applications.

Theory:

Problem definition:

The **Bellman-Ford algorithm** is a classic Dynamic Programming (DP) algorithm used to solve the Single Source Shortest Path (SSSP) problem. While Dijkstra's algorithm is faster, Bellman-Ford is more versatile because it can handle graphs containing **negative edge weights**.

The Dynamic Programming Approach

The algorithm follows the DP principle by breaking the problem into subproblems. It relies on the property that a shortest path between two vertices in a graph with vertices can contain at most *V-1* edges (provided there are no negative cycles).

The subproblem can be defined as: *Let $L_i(v)$ be the shortest path from source s to v vertex using at most i edges.*

Recursive Formula:

$$dist[v] = \min(dist[v], dist[u] + weight(u, v))$$

Negative Cycle Detection:

One of the most important features of Bellman-Ford is its ability to detect **negative weight cycles**. If we relax all edges one more time (the time) and the distance still decreases, it indicates that a negative cycle exists, meaning a "shortest" path is undefined as it could infinitely decrease.

Algorithm:

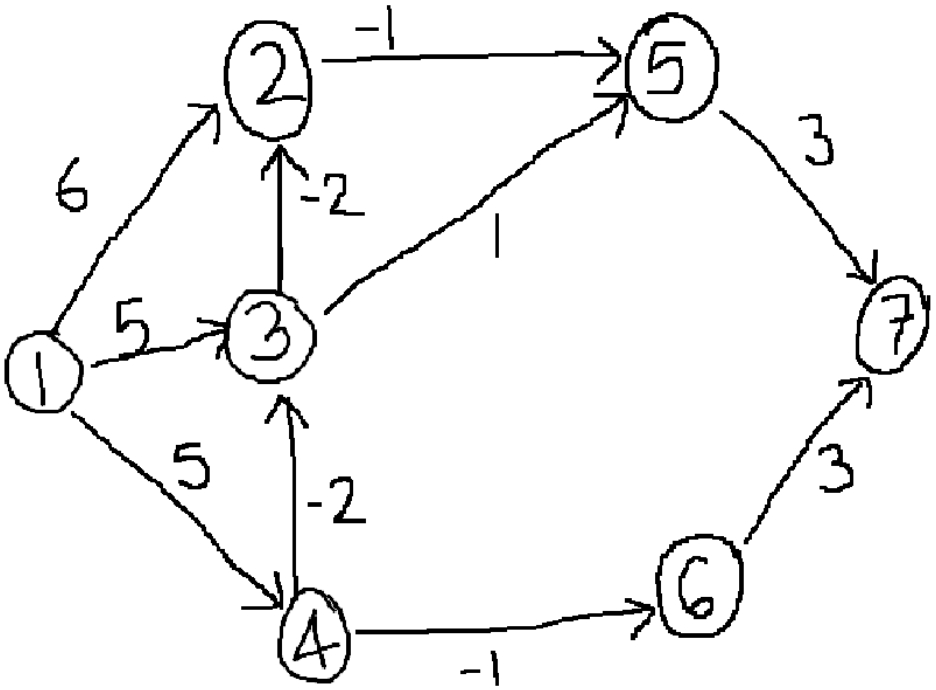
```
Initialize:
  - For each vertex v in V:
      dist[v] = infinity
  - dist[S] = 0

Relax Edges (V - 1 times):
  - For i from 1 to |V| - 1:
      For each edge (u, v) with weight w in E:
        If dist[u] + w < dist[v]:
          dist[v] = dist[u] + w

Check for Negative Cycles:
  - For each edge (u, v) with weight w in E:
      If dist[u] + w < dist[v]:
        Return "Graph contains a negative weight cycle"

Return dist[]
```

Example:



Solution for the example:

		1	2	3	4	5	6	7
Initialization	0	0	inf	inf	inf	inf	inf	inf
k	1	0	6	5	5	inf	inf	inf
	2	0	3	3	5	5	4	inf
	3	0	1	3	5	2	4	5
	4	0	1	3	5	0	4	3
	5	0	1	3	5	0	4	3
	6	0	1	3	5	0	4	3

Analysis of algorithm:

Let
V = 7 vertices.
E = total edges in graph.

Initialization row shows distances.
Source vertex = 1.
d(1)=0. others = ∞.

Iterations run V−1 = 6 times.

k = 1
Edges relaxed once.
Distances update to: 0, 6, 5, 5, ∞, ∞, ∞

k = 2
All edges checked again.
Distances improve: 0, 3, 3, 5, 5, 4, ∞

k = 3
Further relaxation.
Distances: 0, 1, 3, 5, 2, 4, 5

k = 4
More updates.
Distances: 0, 1, 3, 5, 0, 4, 3

k = 5
No change.
Distances stable.

k = 6
No change again.
Final distances same.

Each iteration costs O(E).
Total cost = O(V × E).

After 6 iterations no change occurs.
So no negative cycle exists.

Cycle sums are ≥ 0.
Hence graph has no negative cycle.

CONCLUSION:

I learned how the Bellman–Ford Algorithm computes shortest paths by repeatedly relaxing all edges for V−1 iterations. I understood that its time complexity is O(V*E) since each iteration processes all edges. I also learned that it can handle negative edge weights and can detect negative cycles. Unlike Dijkstra's Algorithm, it does not follow a greedy approach.